

Robust Backtest Protocol for Qullamaggie-Inspired Crypto Perpetual Strategies

Executive summary

This report treats “Qullamaggie-inspired” as a **breakout and episodic-momentum strategy family** whose exact alpha rules are external inputs to the backtest engine. Public material associated with Qullamaggie emphasizes momentum breakouts after prior expansion and consolidation, but the user did **not** specify the exact entry logic, stop logic, pyramiding logic, leverage caps, or portfolio sizing rules. Those strategy parameters are therefore **unspecified** and must remain configurable rather than hard-coded in the research harness. ¹

For crypto perpetuals, the safest production design is **event-driven, exchange-specific, and mark-price aware**. The universe must be reconstructed from historical instrument metadata and delisting notices, not from today’s live contract list. Funding, fees, liquidation, and mark prices must use venue-native rules because Binance, Bybit, and Deribit differ materially in funding settlement frequency, mark-price calculation, risk limits, liquidation path, and insurance-fund / ADL handling. Binance exposes listing timestamps and delisting dates through `exchangeInfo`; Bybit exposes `launchTime`, `deliveryTime`, `fundingInterval`, and status through `instruments-info`; Deribit exposes `creation_timestamp`, `expiration_timestamp`, `state`, and `is_active` through `public/get_instruments`. ²

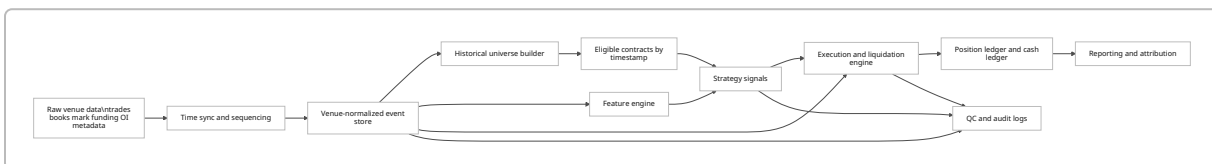
The recommended default is:

Component	Default production configuration
Simulation engine	Event-driven core with sub-second ordering preserved; 1-second state checkpoints for features and reporting
Venue scope	Binance USDⓈ-M, Bybit linear/inverse, Deribit perpetuals; FTX archival only if separately procured and versioned
Universe refresh	Daily eligibility recomputation at 00:05 UTC; weekly watchlist/ranking refresh at Monday 00:05 UTC
Liquidity screen	30-day median daily notional $\geq$ 20M USD ; 30-day median top-of-book spread $\leq$ 15 bps ; median displayed depth inside ± 10 bps $\geq$ 250k USD ; 30-day median open interest $\geq$ 10M USD
Listing rule	No contract-level feature history before official listing timestamp
Funding source of truth	Official historical funding endpoints; mark/ticker streams only as fallback and QC

Component	Default production configuration
Mark-price source of truth	Official historical mark series or venue ticker/mark streams; formula recomputation only for validation
Execution model	Book-walk slippage + latency + queue/partial-fill model + residual impact overlay
Margin mode for research baseline	Isolated, one-way, per-position margin; cross/portfolio margin as separate scenario set
Validation	Rolling outer walk-forward, CPCV inside train only, purging + embargo
Search	Sobol or random coarse search, then Bayesian refinement on top decile candidates
Overfitting gates	Minimum trade count, parameter stability, Deflated Sharpe Ratio > 0, PBO target < 20%
Regimes	Daily 4-state HMM on market-wide volatility, depth, spread, funding, and open-interest features
Reproducibility	Full config hash, Git commit, Docker image digest, data snapshot hash, deterministic seeds

The main engineering principle is simple: **use exchange-published state as truth wherever possible, and only reconstruct what the exchange does not publish**. This is especially important for mark prices, liquidation triggers, and funding mechanics, because venue rules change over time. Binance's mark-price methodology has been revised over time and exchanged via official announcements and stream field changes; Bybit's documentation shows contract-type-specific mark rules and also contains page-to-page differences; Deribit documents a mark price based on index anchoring and EMA smoothing of the premium. The backtest protocol therefore needs a **versioned exchange-rule registry** keyed by `venue × instrument_type × effective_datetime`. ³

One more point is unavoidable: a backtest protocol for a nonstationary, highly optimized strategy family should not rely on a single walk-forward path and a single Sharpe ratio. Bailey and López de Prado's work on backtest overfitting and Deflated Sharpe Ratio, together with later work comparing CPCV against conventional walk-forward schemes, makes a strong case for purging, embargoing, multiple-path validation, and explicit multiple-testing controls. ⁴



Data architecture, universe construction, and survivorship controls

A robust universe for crypto perpetuals has to be reconstructed from **historical instrument state**, not inferred from current listings. Binance publishes `onboardDate`, `deliveryDate`, `status`,

contractType, liquidationFee, and order filters. Bybit publishes launchTime, deliveryTime, status, fundingInterval, price and quantity filters, and delisting time for perpetuals through deliveryTime. Deribit publishes creation_timestamp, expiration_timestamp, is_active, state, instrument IDs, contract size, and related metadata; it also provides instrument-creation notifications and explicit policies for futures introductions. ⁵

The core input schema should be fixed and versioned. A minimum research-grade schema is:

Dataset	Required fields	QC checks
instrument_master	venue, symbol, instrument_id, contract_type, quote_ccy, margin_ccy, contract_size, tick_size, qty_step, min_notional, listing_ts, delist_ts, status, risk_tier_ref, funding_interval_hours, rule_version	No overlapping active intervals for same symbol; listing_ts < delist_ts; filters non-null after listing; rule version present
trade_events	venue_ts, recv_ts, symbol, trade_id, price, qty, side, sequence	Strictly monotone within (venue, symbol) after dedupe; no duplicate (symbol, trade_id)
book_events	venue_ts, recv_ts, symbol, side, price, qty, update_first, update_last, prev_update, cross_seq/change_id	Sequence continuity; no negative qty; book never crossed after applying delta
mark_series	venue_ts, symbol, mark_price, index_price, aux_fields, rule_version	Mark/index positive; stale gap threshold; compare against formula tolerance
funding_history	settlement_ts, symbol, funding_rate, cap, floor, interval_hours, source	Settlement cadence matches rule version; no duplicates per interval
open_interest	ts, symbol, oi, oi_value	Non-negative; cadence stable
corporate_token_events	event_ts, base_asset, event_type, old_symbol, new_symbol, ratio, exchange_announcement_ref	No event without affected instrument map

Dataset	Required fields	QC checks
trading_costs	ts_effective, venue, account_tier, maker_fee, taker_fee, liquidation_fee, settlement_fee, bnb_or_discount_flag	Non-negative or documented rebate; effective periods non-overlapping

Universe construction rules

A production default should build a *daily eligibility set* and a *weekly ranking set*.

Selection rule

- Include only perpetuals whose status is tradeable at timestamp `t`.
- Exclude pre-launch contracts until the exchange says they are tradeable. Bybit explicitly distinguishes `PreLaunch` from `Trading`. Binance and Deribit expose equivalent active-state fields. ⁶
- Use exchange-native listing timestamps as first tradable timestamps:
- Binance: `onboardDate`
- Bybit: `launchTime`
- Deribit: `creation_timestamp` or first active state notification. ⁷

Liquidity thresholds

- Default eligibility thresholds:
- `median_30d_daily_notional_usd >= 20_000_000`
- `median_30d_bid_ask_spread_bps <= 15`
- `median_30d_displayed_depth_usd_within_10bps >= 250_000`
- `median_30d_open_interest_usd >= 10_000_000`
- `days_live >= 30` for full eligibility, with a separate “new listing” bucket if the research team wants IPO-style event trades.
- Range for sensitivity:
- Daily notional: 5M to 100M
- Spread: 5 to 30 bps
- Depth: 100k to 1M USD
- OI: 2M to 50M USD
- Days live: 7 to 60

These thresholds are recommendations, not exchange rules. The user did not specify them, so they should remain configurable.

Rebalance cadence

- Recompute raw eligibility daily at `00:05 UTC`.
- Recompute ranking/watchlist weekly at `Monday 00:05 UTC`.
- Allow intraday event-driven entries/exits only within the current eligible set.

- For breakout families, keep the current week's ranked watchlist frozen except when a contract is delisted, halted, or becomes ineligible on hard safety grounds.

Forks, rebrands, token swaps, airdrops

- Perpetual holders generally do **not** receive synthetic spot entitlements by default. Do **not** credit airdrops or forks to perpetual positions unless the venue's contract announcement explicitly does so.
- If the exchange delists the old perp and lists a new ticker as part of a token swap or rebrand, treat that as a **contract lifecycle event**, not a corporate action on a continuous position.
- Binance and Bybit both announce futures/perpetual delistings around token swaps and typically auto-close positions and delist the old contract rather than transform the open perp position into the new one. 8

Pseudocode

```
def build_daily_universe(ts_utc, instrument_master, liquidity_features,
token_events):
    active = []
    for inst in instrument_master:
        if ts_utc < inst.listing_ts:
            continue
        if inst.delist_ts is not None and ts_utc >= inst.delist_ts:
            continue
        if inst.status not in {"TRADING", "Trading", "open"}:
            continue
        lf = liquidity_features.get(inst.symbol, ts_utc.date())
        if lf is None:
            continue
        if lf.days_live < cfg.min_days_live:
            continue
        if lf.median_30d_daily_notional_usd < cfg.min_daily_notional_usd:
            continue
        if lf.median_30d_spread_bps > cfg.max_spread_bps:
            continue
        if lf.median_30d_depth_usd_10bps < cfg.min_depth_usd_10bps:
            continue
        if lf.median_30d_open_interest_usd < cfg.min_open_interest_usd:
            continue

        # token-event gate
        evt = token_events.get_active_event(inst.base_asset, ts_utc)
        if evt is not None and cfg.exclude_rebrand_windows:
            if abs((evt.event_ts - ts_utc).total_seconds()) <=
cfg.rebrand_blackout_sec:
                continue
```

```
active.append(inst.symbol)
return sorted(active)
```

Survivorship-bias controls

The backtest must reconstruct the *historical eligible set at each timestamp* and then store it as a first-class artifact. Do not rebuild it on the fly from later metadata.

The inclusion / exclusion rules should be:

- Include every contract that was listed and tradeable during the sample, even if later delisted.
- Include delisted contracts up to the exact exchange delisting timestamp.
- Never backfill contract-specific features before listing.
- Never remove a contract from old dates because it is absent today.
- Exclude permanently non-tradeable prelaunch instruments and synthetic test symbols.
- Exclude intervals marked with unrecoverable data corruption or unresolved sequence gaps; do not silently forward-fill them.

A simple audit is to compare a **survivorship-free run** against a **naive current-live-universe run** and report the performance delta as “survivorship bias impact.” This should be a required report item, not an optional appendix. The reason is standard: survivorship materially inflates backtests when failed or delisted names are dropped. The broader backtesting literature treats survivorship bias and look-ahead as core distortions, and the same logic applies to fast-rotating crypto perp listings. ⁹

Decision tree for missing or ambiguous instrument history

- Missing `listing_ts`, but first trade exists
→ infer provisional `listing_ts = first_trade_ts`, mark `metadata_inferred=True`, include only if explicit exchange status becomes tradeable within 24h.
- Missing `delist_ts`, but status flips from tradeable to inactive/delivered
→ set `delist_ts = first_nontradeable_ts`.
- Rebrand old/new symbols overlap
→ treat as two instruments unless the exchange explicitly declares position transfer. Default is **no transfer**.
- Instrument metadata changes mid-life
→ create new `rule_version` interval from effective timestamp; preserve both periods.

Complexity and storage

For five years, 100 contracts, 1-second dense checkpoints:

- total seconds $\approx$ **157,788,000**
- contract-seconds $\approx$ **15.78 billion**

A dense 1-second core panel with about 80 bytes per row is about **1.26 TB raw** and typically **250–500 GB compressed** in a columnar store. A 1-second top-50 book snapshot is an order of magnitude larger and should usually be stored as event deltas plus derived microstructure features, not as full dense snapshots. This is an engineering estimate from row counts and field widths.

Merit of this design

The universe layer is no longer a convenience filter. It becomes a deterministic historical object that every train / validation / test fold consumes identically. That is the only way to separate true alpha from listing turnover, delisting contamination, and reverse survivorship effects.

Contract lifecycle, funding, fees, slippage, and liquidation

The lifecycle layer must be venue-specific. Binance, Bybit, and Deribit do not agree on delisting settlement, funding cadence, mark-price construction, or liquidation sequencing. Binance delisting uses the average price index over the last 30 minutes before delisting and charges settlement fees equal to taker fees; trading restrictions begin before delisting and users are pushed reduce-only in the final window. Bybit delisting announcements likewise auto-cancel orders and auto-close open positions, typically at an average index price over the 30 minutes before delisting. Deribit differs because perpetuals do not expire, while dated futures follow formal introduction and expiry schedules; instrument states move through `open`, `settlement`, `delivered`, `inactive`, `halted`, and `archived`. ¹⁰

Lifecycle treatment

- **Perpetuals**
 - No roll in the ordinary sense.
 - Position remains in same contract until exit, liquidation, or delisting.
 - If delisting occurs mid-trade, settle exactly per venue announcement or venue generic rule.
- **Dated futures**
 - Maintain a separate roll module.
 - Recommended default roll trigger: earliest of
 - `days_to_expiry <= 3`
 - `next_contract_oi > front_contract_oi`
 - `next_contract_depth_10bps > front_contract_depth_10bps`
 - Roll execution should be modeled as two linked child orders, with spread/impact on both legs.

Backfill policy

- Contract-specific microstructure, funding, open interest, and fee history: **no backfill before official listing**.
- Underlying spot/index features used as macro context may exist before listing, but they must be stored as separate datasets and must not fabricate pre-listing contract tradability.

Orphaned positions

- If a venue delists or administratively closes a contract while a position is open:
- close at official settlement method,

- cancel resting orders,
- realize funding/fees due up to settlement cut-off,
- forbid symbol re-entry unless a new instrument is listed.

Pseudocode

```
def process_contract_lifecycle(event, portfolio, rules):
    inst = rules.instrument(event.symbol, event.ts)

    if event.type == "DELIST_NOTICE":
        portfolio.flags[event.symbol].delist_ts = event.delist_ts
        portfolio.flags[event.symbol].reduce_only_from = event.reduce_only_ts

    if event.type == "CLOCK" and portfolio.has_position(event.symbol):
        if event.ts >= portfolio.flags[event.symbol].reduce_only_from:
            cancel_non_reduce_only_orders(event.symbol)

        if event.ts >= portfolio.flags[event.symbol].delist_ts:
            settle_price = compute_delist_settlement_price(
                symbol=event.symbol,
                venue=inst.venue,
                rule_version=inst.rule_version,
                ts=event.ts,
            )
            close_position(event.symbol, settle_price,
                reason="DELIST_SETTLEMENT")
            cancel_all_orders(event.symbol)
            disable_symbol(event.symbol)
```

Funding modeling

Binance funding is a periodic transfer between longs and shorts; the default interval is 8 hours, but Binance can alter intervals under volatility and now publishes adjusted funding interval information through `fundingInfo`. Funding is applied only if a position is open at the funding timestamp, with a documented small transaction-time deviation window. Bybit calculates funding from an interest-rate component and a premium-index component, updating every minute and usually settling every 8 hours, though the instrument metadata must be checked per symbol. Deribit's funding is a continuous accrual concept with payment derived from mark-index premium, a damper, a cap, and time fraction; realized settlement is moved at daily settlement, with formula details documented explicitly. ¹¹

Funding requirements

- Store official funding history as the source of truth.
- Store per-symbol interval/cap/floor history.
- Apply funding on the exact exchange settlement calendar, not on an assumed uniform 8-hour cadence.

- For Binance, honor dynamic interval changes from `fundingInfo`.
- For Deribit, model continuous accrual at the finest available resolution if the strategy holds positions through settlement cycles.

Decision tree for missing funding

- Official settled funding present
→ use it.
- Official settlement missing, but pre-settlement mark/ticker snapshot exists within tolerance and venue formula is stable for that period
→ use provisional reconstructed funding, set `funding_imputed=True`, include only in sensitivity run.
- Missing official settlement and no reliable reconstruction
→ freeze new entries on that symbol, keep exits enabled, and invalidate the primary run if missing funding exposure exceeds a configured NAV threshold, default **5 bps of fold NAV**.

Fees

Binance charges maker/taker fees by user tier and exposes symbol commission rates via `commissionRate`; delisting settlement fees equal taker fees; liquidation clearance fees exist separately. Bybit publishes taker and maker fees by contract family and also applies a fixed delisting fee of 0.05% on positions closed because of delisting. Deribit uses a maker-taker model, with baseline perpetual/futures fees around 0 / 5 bps for many futures/perpetual products and exchange-published liquidation fees on top during liquidation. ¹²

Fee modeling requirements

- Every execution record must include:
 - venue
 - account tier / discount regime
 - maker/taker flag
 - fee schedule version
 - base fee
 - discount or rebate
 - liquidation or settlement surcharge
- Discount tokens or venue-specific payment schemes are **unspecified by the user**; default them to **off** in base runs, and run a separate discount-enabled sensitivity if relevant.

Dynamic slippage model

The default should be a **hybrid execution model**: 1. Apply latency to determine the actionable book. 2. Book-walk through visible depth until the child order is filled or the modeled visible book is exhausted. 3. If displayed depth is insufficient, apply a residual impact overlay tied to order size, recent volatility, and participation rate. 4. Model partial fills and missed fills separately.

This choice is better than fixed-bps slippage because the exchanges publish rich book state, update IDs, and top-of-book changes at sub-second frequencies, while market microstructure research strongly suggests execution cost scales with available liquidity and order size rather than a constant number of basis points. Almgren–Chriss provides the canonical execution-cost decomposition; Bouchaud/Farmer/Lillo and later impact work support impact sensitivity to order flow and liquidity state. ¹³

Default slippage equations

- Let Q be child order notional.
- Let $\text{book_walk_px}(Q, t + \text{latency})$ be the VWAP from visible depth after latency.
- Let depth_10bps be visible notional depth inside ± 10 bps.
- Let sigma_1m be one-minute realized volatility in bps.
- Let $\text{part_1m} = Q / \max(\text{ADV_1m}, \text{eps})$.

Then:

```
base_exec_px = book_walk_px(Q, t + latency_ms)
residual_impact_bps = lambda1 * max(0, (Q - depth_10bps) / max(depth_10bps,
eps)) * spread_bps
                    + lambda2 * sigma_1m * sqrt(max(part_1m, 0))
final_exec_px = base_exec_px * (1 + side_sign * residual_impact_bps / 10_000)
```

Recommended defaults:

- $\text{latency_ms} = 150$ in base run; sensitivity $\{50, 150, 500\}$
- $\text{lambda1} = 0.50$
- $\text{lambda2} = 1.00$
- child order participation cap per second: $\min(5\%, 10\% \text{ of displayed depth})$
- residual fill timeout: 1 second for aggressive IOC-style execution, 30 seconds for passive breakout entry slices

These are engineering defaults, not exchange rules.

Alternative slippage designs

Model	Strength	Weakness	Recommended use
Fixed bps	Fast, easy	Unrealistic in thin names and news spikes	Never as primary model
Spread-only	Better than fixed bps	Misses depth depletion and volatility regime	Quick smoke tests only
Full book-walk	Venue-faithful	Needs high-quality books, heavier compute	Primary short-horizon execution model

Model	Strength	Weakness	Recommended use
Book-walk + residual impact	Best compromise	More parameters	Recommended default
Queue-position simulator	Most realistic for passive orders	Complex, calibration-heavy	Advanced maker strategy research only

Liquidation and mark-price modeling

This part should use **official mark series as the source of truth**. Recompute the formula only for QC, because venue formulas can change or vary by contract class. Binance uses mark price for liquidation and publishes mark-price streams as well as rule updates. Bybit states that mark price is used for liquidation and gives explicit contract formulas, but its documentation also varies across pages and contract categories. Deribit documents mark-price construction using index anchoring and EMA smoothing of premium, with explicit notes that mark price may fall outside the bid-ask spread and should not be used as a trading signal. ¹⁴

Venue-specific liquidation model

- **Binance**

- Liquidation triggered when collateral falls below maintenance margin.
- Uses mark price, smart liquidation, and tries to reduce deficit through a large IOC order before bankruptcy handling.
- Insurance fund absorbs bankrupt-position losses where possible; ADL is the tail mechanism. ¹⁵

- **Bybit**

- Liquidation uses maintenance margin rate and bankruptcy price.
- If the liquidation engine cannot close better than bankruptcy price, the insurance fund covers deficit; ADL follows if the fund is insufficient.
- Bybit exposes all-liquidation feeds and explicit ADL rules. ¹⁶

- **Deribit**

- Incremental liquidation closes positions until maintenance margin falls below margin balance.
- On portfolio margin, the liquidation process can include delta-hedging via futures/perpetuals.
- Liquidation fees accrue to the insurance fund; negative post-liquidation equity can be covered by the insurance fund. ¹⁷

Backtest liquidation algorithm

- Baseline research mode: isolated margin, one symbol = one liquidation path.
- Trigger on official `mark_price`, not last trade.
- Recalculate maintenance margin after each partial liquidation chunk.
- Use exchange-specific close fees and bankruptcy logic.
- Record liquidation as an execution with reason code and exchange rule version.
- Apply insurance-fund / ADL effects as follows:
- base run: exchange-documented logic
- stress run: worsen close price by an additional adverse jump equal to $\max(1 \text{ spread}, 0.5 * \text{sigma}_{1m})$ when liquidation feed intensity is extreme

Pseudocode

```
def maybe_liquidate(position, account, market_state, rules):
    mark = market_state.mark_price[position.symbol]
    mm = compute_maintenance_margin(position, rules, mark)
    equity = compute_position_equity(position, mark, account)

    if equity >= mm:
        return

    if position.venue == "BINANCE":
        qty_to_liq = smart_liq_chunk(position, rules)
        px = ioc_book_walk_after_latency(position.side_to_close(), qty_to_liq)
        apply_exec(position, qty_to_liq, px, liquidation=True)
        if compute_position_equity(position, mark, account) <
compute_maintenance_margin(position, rules, mark):
            transfer_bankrupt_stub_to_insurance_or_adl(position, mark, rules)

    elif position.venue == "BYBIT":
        while compute_position_equity(position, mark, account) <
compute_maintenance_margin(position, rules, mark):
            qty_to_liq = laddered_liq_chunk(position, rules)
            px = bankruptcy_or_market_close_px(position, qty_to_liq, mark,
rules)

            apply_exec(position, qty_to_liq, px, liquidation=True)
            if insurance_fund_shortfall(position.symbol, rules):
                trigger_adl(position.symbol)

    elif position.venue == "DERIBIT":
        while compute_position_equity(position, mark, account) <
compute_maintenance_margin(position, rules, mark):
            target_instr = choose_highest_margin_contributor(account, rules)
            qty_to_liq = incremental_chunk(target_instr, rules)
            px = ioc_book_walk_after_latency(target_instr.side_to_close(),
qty_to_liq)

            apply_exec(target_instr, qty_to_liq, px, liquidation=True)
            if rules.margin_mode == "portfolio" and still_underwater(account):
                maybe_open_delta_hedge(account, rules)
```

Timestamp integrity and event ordering

The event layer is where many backtests quietly fail. Binance book streams expose event time `E`, transaction time `T`, and update IDs `U/u/pu`. Bybit exposes `ts`, update ID `u`, and cross sequence `seq`, plus server time down to nanoseconds through its time endpoint and NTP guidance in its API guide. Deribit exposes order-book `change_id` and `prev_change_id`, request microsecond timestamps

`usIn/usOut/usDiff`, and a single matching engine behind multiple gateway nodes. That means a serious simulator has enough sequence information to reconstruct order. It should use it. ¹⁸

The required timestamp schema for every raw event is:

Field	Meaning
<code>venue_ts</code>	Exchange-published event or transaction timestamp
<code>recv_ts</code>	Local gateway receive timestamp
<code>norm_ts</code>	Internal normalized timestamp after drift correction
<code>seq_primary</code>	Venue-native ordering field (<code>u</code> , <code>change_id</code> , etc.)
<code>seq_secondary</code>	Cross-stream tie-breaker (<code>pu</code> , <code>seq</code> , <code>trade_id</code> , <code>recv_order</code>)
<code>source_channel</code>	trade, book, mark, funding, liquidation, metadata
<code>clock_offset_us</code>	Measured drift between local and venue time
<code>reordered_flag</code>	whether late arrival required resequencing

Hard requirements

- All data stored in UTC.
- All local collectors PTP- or NTP-synchronized; max local drift budget: **1 ms** for historical collection, **100 µs** if the desk later uses the same stack for live trading.
- Never downsample to 1-second bars before you reconstruct sequencing.
- For historical 1-second backtests, preserve raw ordering inside each 1-second bucket and then checkpoint state at the bucket close.

Ordering rules

- **Binance**: apply book events only if sequence continuity holds; reject or resync on update-ID gaps. ¹⁹
- **Bybit**: reset on new snapshot; order within symbol by `seq` , then `u` , then `ts` , then `recv_ts` . ²⁰
- **Deribit**: verify `prev_change_id == last_change_id` ; otherwise resubscribe or gap-fill from snapshot. Use `usIn/usOut` to diagnose network lag on request-response channels. ²¹

Decision tree for sequence problems

- Small gap, snapshot available within retry budget
→ reset from latest snapshot, continue, flag gap.
- Large gap or repeated reset on same symbol
→ mark interval untrustworthy; disable passive fill modeling on that symbol until recovery.

- Venue timestamp arrives out of order but sequence is consistent
→ respect sequence, not wall-clock.
- No sequence field on a channel
→ order by `venue_ts`, then `recv_ts`, then monotone ingest counter.

Pseudocode

```
def sequence_events(raw_events):
    grouped = group_by_symbol_and_channel(raw_events)
    ordered = []

    for key, events in grouped.items():
        if key.venue == "BINANCE" and key.channel == "book":
            events.sort(key=lambda e: (e.U, e.u, e.recv_ts))
            ordered.extend(apply_binance_gap_rules(events))
        elif key.venue == "BYBIT" and key.channel == "book":
            events.sort(key=lambda e: (e.seq, e.u, e.ts, e.recv_ts))
            ordered.extend(apply_bybit_snapshot_delta_rules(events))
        elif key.venue == "DERIBIT" and key.channel == "book":
            events.sort(key=lambda e: (e.change_id, e.recv_ts))
            ordered.extend(apply_deribit_prev_change_rules(events))
        else:
            events.sort(key=lambda e: (e.venue_ts, e.recv_ts, e.ingest_idx))
            ordered.extend(events)

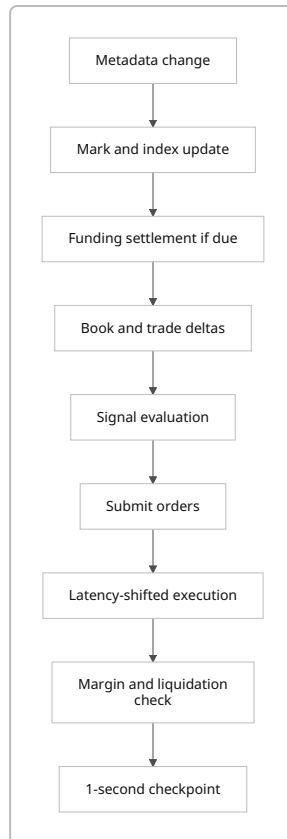
    return global_merge(ordered, key=lambda e: (e.norm_ts, e.priority,
e.recv_ts, e.ingest_idx))
```

Event-engine order of operations

At any time `t`, the engine should process events in this order:

1. metadata / lifecycle changes
2. mark/index updates
3. funding settlement events
4. order-book and trade events
5. strategy signal evaluation
6. order submission
7. execution simulation after latency
8. margin and liquidation check
9. end-of-second checkpoint

This order reduces look-ahead contamination between mark, fills, and liquidation triggers.



Computational implications

A pure 1-second bar simulator is $O(T \cdot N)$. An event-driven simulator is closer to $O(E + O_{\text{exec}} \cdot D)$, where E is the number of raw events, O_{exec} the number of orders, and D the number of consumed book levels per order. For breakout/swing strategies, the event-driven cost is worth it because the most important errors come from execution timestamps and mark-price liquidation, not from daily signal logic.

Walk-forward validation, parameter search, and regime segmentation

The validation design should separate three distinct problems:

- **research fit:** find admissible parameter regions;
- **selection bias:** avoid picking the best of many noisy trials;
- **nonstationarity:** test whether the strategy survives regime shifts.

Standard single-path walk-forward is insufficient for all three. Bailey et al. show why repeated model search inflates in-sample edge; the Deflated Sharpe Ratio corrects selection bias and non-normal returns; López de Prado's purging and embargo ideas are designed to stop leakage; later work comparing CPCV with ordinary walk-forward reports lower PBO and more stable out-of-sample distributions for the combinatorial purged approach. ⁴

Recommended default validation stack

- **Outer loop:** rolling walk-forward for chronology and drift
- train 18 months
- validation 3 months
- test 3 months
- step 3 months
- **Inner loop:** CPCV or purged k-fold on the training block only
- $\text{purge} = \max(\text{label_horizon}, \text{max_hold_period})$
- embargo = 1 day minimum, or $\max(\text{purge}, \text{funding_interval_max})$ if holding periods are intraday
- **Final holdout:** last 6 months untouched until the research package is frozen

This is a recommendation. The user did not specify the holding period, so the purge and embargo lengths should be derived from the strategy's actual label horizon once that is defined.

Look-ahead prevention rules

- Features at time t may only use data with $\text{event_ts} \leq t$.
- Daily liquidity thresholds used for a session starting at 00:05 UTC may only use data ending at 00:00 UTC.
- Funding used in decision features may use the last known published estimate, but realized funding PnL is applied only at settlement time.
- Delist notices and rule changes must only become visible when the venue published them or the instrument metadata changed.

Parameter search design

A good default is **coarse random / Sobol search**, then **local Bayesian refinement**.

- **Grid search**
 - good for low-dimensional interpretable rule sets
 - wasteful in high dimension
- **Random / Sobol**
 - covers high dimensions better at fixed budget
 - easy to parallelize
- **Bayesian**
 - good after a coarse map exists
 - fragile under noisy, regime-shifting objective surfaces

For this strategy family, the best practical sequence is: 1. Sobol or random search over all admissible parameters. 2. Filter by hard constraints. 3. Bayesian refinement around top-decile candidates. 4. Select by **robustness score**, not raw Sharpe.

Hard constraints

- minimum 150 trades in pooled train+validation per candidate, unless the strategy is explicitly slower
- max turnover cap

- max liquidation count per year
- max single-symbol concentration
- minimum median daily depth coverage at entry/exit
- parameter monotonicity or plausible bounds
- no candidate selected if it fails DSR or PBO gates

Robustness score A good engineering score is:

```
score =
  0.35 * median(test_sharpe_across_paths)
+ 0.20 * median(test_sortino_across_paths)
+ 0.15 * pnl_stability_score
+ 0.10 * parameter_stability_score
+ 0.10 * slippage_robustness_score
+ 0.10 * latency_robustness_score
- penalties_for_liquidations_and_data_imputation
```

Regime segmentation

Regimes should be built from *lagged market-state variables*, not future labels. Good daily features are:

- BTC and market-wide realized vol: 1d, 7d, 30d
- cross-sectional median spread
- cross-sectional median depth at 10 bps
- cross-sectional open-interest growth
- funding dispersion and funding sign breadth
- cross-sectional breakout participation rate
- stablecoin or quote-currency stress proxies if available
- exchange outage or extraordinary notice flags

The recommended default is a **4-state Gaussian HMM** on daily standardized features. The reason is not that HMMs are theoretically perfect; it is that they impose persistence, are easy to retrain inside each training window, and map well to “calm trend / calm range / volatile trend / distressed illiquidity” states. This is an engineering recommendation informed by the fact that volatility, order-flow, and liquidity conditions shape impact and basis behavior in perpetuals. Academic work on perpetuals and microstructure supports the importance of funding, basis, and liquidity state in price formation. ²²

Regime-aware training rules

- Fit regimes only on the training block.
- Predict regime labels forward into validation/test using the fitted model.
- Allow either:
 - regime-conditioned parameters, or
 - one parameter set with regime-conditioned exposure caps
- Never use future regime labels in training features.

Pseudocode

```

def walk_forward_schedule(start_ts, end_ts, train_months=18, val_months=3,
test_months=3, step_months=3):
    windows = []
    cur = start_ts
    while cur + months(train_months + val_months + test_months) <= end_ts:
        train = (cur, cur + months(train_months))
        val = (train[1], train[1] + months(val_months))
        test = (val[1], val[1] + months(test_months))
        windows.append({"train": train, "val": val, "test": test})
        cur = cur + months(step_months)
    return windows

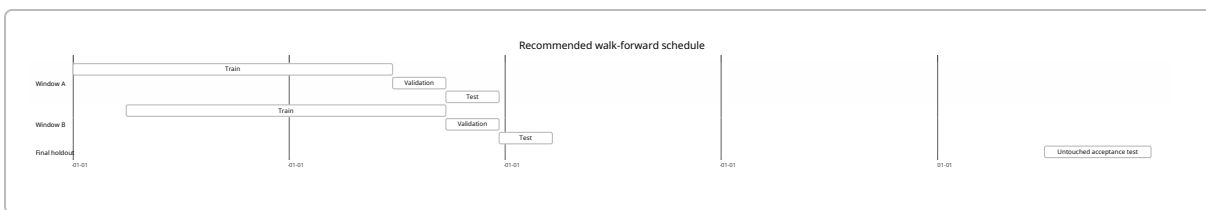
def select_parameters(train_data, label_horizon):
    folds = cpcv_splits(train_data, purge=label_horizon,
embargo=max(label_horizon, ONE_DAY))
    candidates = sobol_sample(space=cfg.param_space, n=cfg.random_budget)
    scored = []
    for p in candidates:
        metrics = evaluate_across_folds(train_data, folds, p)
        if violates_hard_constraints(metrics):
            continue
        scored.append((p, robustness_score(metrics)))

    top = top_decile(scored)
    refined = bayesian_refine(top, budget=cfg.bayes_budget)
    final = choose_by_score_dsrandpbo(refined)
    return final

```

Alternative validation designs

Design	Strength	Weakness	Default decision
Anchored walk-forward	More train data later in sample	Can hide drift and overweight old regimes	Good secondary check
Rolling walk-forward	Better for nonstationarity	Less data per window	Primary outer loop
Purged k-fold only	Efficient	Loses full chronology	Inner loop only
CPCV	Better distributional view, lower selection bias	More computation	Primary inner loop for finalists
Single holdout	Cheap	Too fragile	Never alone



Reporting metrics, tests, audit logs, and reproducibility

The reporting layer should be bifurcated into **trade-level truth** and **portfolio-level aggregation**. The engine should never compute only summary performance. It must retain a deterministic trade ledger and a deterministic cash ledger.

Required reporting outputs

Trade-level

- entry/exit timestamps
- symbol, venue, side, signal family
- intended size vs filled size
- entry/exit prices: decision, submitted, book-walk, final fill
- gross PnL
- net PnL
- slippage
- maker/taker fees
- funding paid/received
- liquidation flag
- delist-settlement flag
- latency bucket
- regime label at entry and exit

Portfolio-level

- cumulative PnL and equity
- realized and unrealized PnL
- gross and net returns
- max drawdown and duration
- Sharpe and Sortino
- tail metrics: 95% / 99% daily VaR and CVaR
- turnover
- exposure, concentration, and leverage
- margin utilization
- liquidation count and liquidation loss
- fee attribution
- funding attribution
- slippage attribution
- survivorship-bias impact

- latency sensitivity
- parameter sensitivity
- data-imputation impact

Attribution requirements

- $\text{net_pnl} = \text{alpha_pnl} - \text{trading_fees} - \text{funding_costs} - \text{settlement_fees} - \text{liquidation_fees} - \text{slippage_costs}$
- report each term separately by:
 - symbol
 - venue
 - regime
 - signal family
 - train/val/test fold

Unit and integration tests

The test suite should mix tiny deterministic fixtures with replay-based integration tests.

Test case	Type	Expected output
Listing timestamp gate	Unit	Symbol excluded before <code>listing_ts</code> , included at or after <code>listing_ts</code>
Delist mid-trade on Binance	Integration	Position closed at official delist settlement price, all orders canceled, no re-entry
Bybit delist due to rebrand	Integration	Old symbol closed and disabled; new symbol treated as distinct contract
Missing funding for one interval	Unit	Base run flags symbol; sensitivity run imputes per fallback rule
Binance depth gap <code>pu != prev_u</code>	Unit	Order book resync required; passive fills disabled until resync
Bybit fresh snapshot after delta stream	Unit	Local book fully reset, not incrementally patched
Deribit <code>prev_change_id</code> mismatch	Unit	Gap flag and snapshot refresh
Partial fill under latency	Integration	Fill quantity < desired quantity; residual order status logged
Liquidation trigger on mark, not last price	Unit	No liquidation if last price crosses but mark does not; liquidation when mark crosses
Deribit incremental liquidation	Integration	Position reduced in chunks until equity $\geq$ maintenance margin or exhausted

Test case	Type	Expected output
CPCV purge/embargo	Unit	No overlapping labels between train and test folds
Survivorship audit	Regression	Naive current-universe run outperforms or differs materially from reconstructed-universe run; delta reported

Expected tolerances

- arithmetic PnL reconciliation tolerance: $1e-8$ in settlement currency
- event ordering reproducibility: exact
- stochastic search reproducibility: exact under fixed seed and deterministic scheduler
- floating-point replay tolerance in portfolio-level summaries: $\leq 1e-10$ relative where feasible

Complexity and storage matrix

Topic	Time complexity	Marginal storage
Universe construction	$O(N_{\text{daily}} \log N_{\text{daily}})$ per refresh	tiny; daily eligibility table
Survivorship reconstruction	$O(M \log M)$ for metadata intervals	tiny to small
Listing/delisting handling	$O(\text{events})$	tiny
Funding application	$O(F)$ where F is funding settlements	tiny
Fee modeling	$O(\text{trades})$	tiny
Slippage / book walk	$O(D)$ per order, D = consumed depth levels	moderate if storing replay details
Liquidation engine	$O(L \cdot D)$ per liquidation episode	moderate
Event sequencing	$O(E \log E)$ globally or linear after partitioning	large raw event store
Walk-forward training	$O(W * P * T_{\text{window}})$ approx.	model artifacts only
Regime segmentation	$O(W * T_{\text{window}} * K * I)$ for HMM/GMM-style fits	small
Reporting	$O(\text{trades} + \text{checkpoints})$	moderate
Audit logs	$O(\text{all key events})$	moderate

For five years and 100 contracts at 1-second checkpoints, the dense core state is about **15.78 billion contract-seconds**. At ~80 bytes/row the raw core panel is about **1.26 TB**; with compression, roughly **250–500 GB**. Full high-depth order-book storage is far larger and should be event-delta based.

Logging and audit requirements

Every run must emit:

- `run_id`
- UTC start/end times
- Git commit hash
- Docker image digest or environment lockfile hash
- Python / compiler / library versions
- CPU architecture
- random seeds for:
 - parameter search
 - any bootstrap or Monte Carlo
 - any tie-breaking
- exact config file
- exact universe snapshot hash
- exact data snapshot hashes by dataset
- rule-version registry hash
- counts of:
 - dropped events
 - sequence gaps
 - funding imputations
 - delist settlements
 - liquidations
 - partial fills
 - fold-level metrics
- parameter candidates explored
- winning candidate rationale
- exception log with symbol and timestamp

Reproducibility controls

- deterministic sorting whenever timestamps tie
- no unordered map iteration in state transitions
- seeds fixed in config and stored in output
- all timestamps normalized to UTC in persisted artifacts
- immutable raw-data layer; normalized layers are versioned, not overwritten
- research notebooks are not authoritative; only pipeline outputs with config hash are authoritative

Default production-ready recommendation

The quant engineering team should ship one **authoritative backtest mode** and several **named sensitivity modes**.

Authoritative mode

- event-driven simulator

- official mark price as truth
- official funding history as truth
- reconstructed historical universe
- isolated margin baseline
- book-walk plus residual impact slippage
- rolling walk-forward outer loop
- CPCV inner loop
- DSR / PBO selection controls
- regime-conditioned exposure caps, not separate alpha logic

Sensitivity modes

- ◦ fixed 5 / 10 / 20 bps simple slippage for sanity bounds
- ◦ latency at 50 / 150 / 500 ms
- ◦ reduced depth shocks
- ◦ funding-imputation stress
- ◦ survivorship-naïve comparison
- ◦ cross margin / portfolio margin scenario
- ◦ venue outage / gap stress

This defaults to a conservative interpretation of what a research backtest should do: it should be less flattering than the live-marketing version, because its job is to survive contact with the exchange rulebook.

Open questions and limitations

Some important details remain **unspecified by the user** and should stay configurable:

- exact Qullamaggie-inspired alpha rules
- holding period and label horizon
- stop-loss and trailing-stop logic
- pyramiding rules
- per-trade risk budget
- leverage cap
- whether the target deployment is retail API, colocated API, or internal low-latency stack
- whether cross margin or portfolio margin is part of the intended live strategy

One venue-specific caveat matters. FTX is historically relevant, but in 2026 it is a bankrupt venue and should be treated as archival-only rather than a live primary design reference. The user asked to prioritize FTX archival materials, but exact archival data procurement and rule-versioning for FTX are not specified in the request. Any FTX inclusion should therefore be a separate archival module with explicit vendor provenance and its own rule registry. ²³

The sharpest unresolved tension is this: the more realistic the simulator becomes, the less any single backtest path matters. With crypto perpetuals, the failure modes are not abstract statistical sins. They are specific timestamps, specific delisting notices, specific mark-price moves, specific book-sequence gaps, and specific moments when the exchange stops behaving like a smooth chart and starts behaving like a clearinghouse.

1 3 TIMELESS setups that have made me TENS OF MILLIONS!

https://quillamaggie.com/my-3-timeless-setups-that-have-made-me-tens-of-millions/?utm_source=chatgpt.com

2 5 7 Exchange Information | Binance Open Platform

<https://developers.binance.com/docs/derivatives/usds-margined-futures/market-data/rest-api/Exchange-Information>

3 Updates on Mark Price Calculation for USD[Ⓢ]-M and COIN-M Futures Contracts (2024-04-03) | Binance Support

<https://www.binance.com/en/support/announcement/detail/30ef571caf2c487dbac20f63a2a49949>

4 The Probability of Backtest Overfitting

https://papers.ssrn.com/sol3/papers.cfm?abstract_id=2326253&utm_source=chatgpt.com

6 Get Instruments Info | Bybit API Documentation

<https://bybit-exchange.github.io/docs/v5/market/instrument>

8 Binance Will Support the Stratis (STRAX) Token Swap and ...

https://www.binance.com/az-AZ/support/announcement/detail/2e6eb8e644664a6ca7b6659a226c41b0?utm_source=chatgpt.com

9 Seven Sins of Quantitative Investing

https://hudsonthames.org/wp-content/uploads/2022/01/DB-201409-Seven_Sins_of_Quantitative_Investing.pdf?utm_source=chatgpt.com

10 Delisting of Futures Contracts | Binance Futures,Binance Futures Delisting,What is Delisting

<https://www.binance.com/en/support/faq/detail/dd60dfbf654d4055aa6b217ea6d5ddba>

11 Introduction to Binance Futures Funding Rates | Binance Futures,What is funding rate,Binance Futures Funding Rates

<https://www.binance.com/en/support/faq/detail/360033525031>

12 User Commission Rate | Binance Open Platform

<https://developers.binance.com/docs/derivatives/usds-margined-futures/account/rest-api/User-Commission-Rate>

13 Diff Book Depth Streams | Binance Open Platform

https://developers.binance.com/docs/derivatives/usds-margined-futures/websocket-market-streams/Diff-Book-Depth-Streams?utm_source=chatgpt.com

14 Mark Price Stream | Binance Open Platform

<https://developers.binance.com/docs/derivatives/usds-margined-futures/websocket-market-streams/Mark-Price-Stream>

15 Binance Futures Liquidation Protocols | Futures last price,Futures mark price,Binance futures liquidation,Mark price vs last price

<https://www.binance.com/en/support/faq/detail/360033525271>

16 Maintenance Margin (USDT Perpetual and Expiry Contracts) - Help Center

<https://www.bybit.com/en/help-center/article/Maintenance-Margin-USDT-Contract>

17 Liquidations – Deribit Support

<https://support.deribit.com/hc/en-us/articles/25944769313309-Liquidations>

18 19 Partial Book Depth Streams | Binance Open Platform

https://developers.binance.com/docs/derivatives/usds-margined-futures/websocket-market-streams/Partial-Book-Depth-Streams?utm_source=chatgpt.com

20 Orderbook | Bybit API Documentation

<https://bybit-exchange.github.io/docs/v5/websocket/public/orderbook>

21 Deribit API Notifications and Subscriptions - Real-Time Data Guide

<https://docs.deribit.com/articles/notifications>

22 Fundamentals of Perpetual Futures

https://papers.ssrn.com/sol3/papers.cfm?abstract_id=4301150&utm_source=chatgpt.com

23 FTX Cryptocurrency Exchange

https://claims.ftx.com/?utm_source=chatgpt.com